

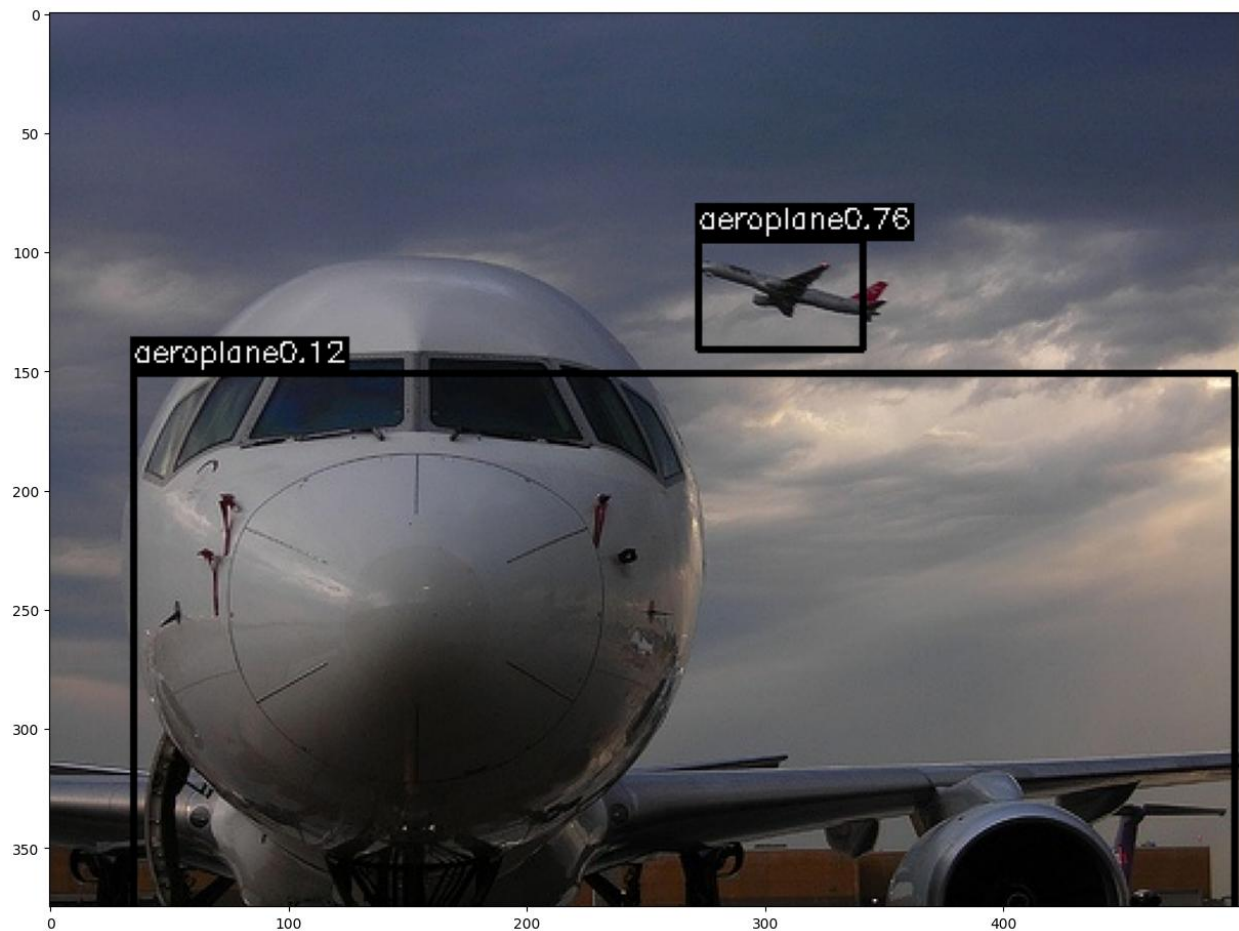
Assignment 4

Name(s): Licheng Xu, Libin Wang

NetID(s): lx8, libin2

Object Detection by YOLO

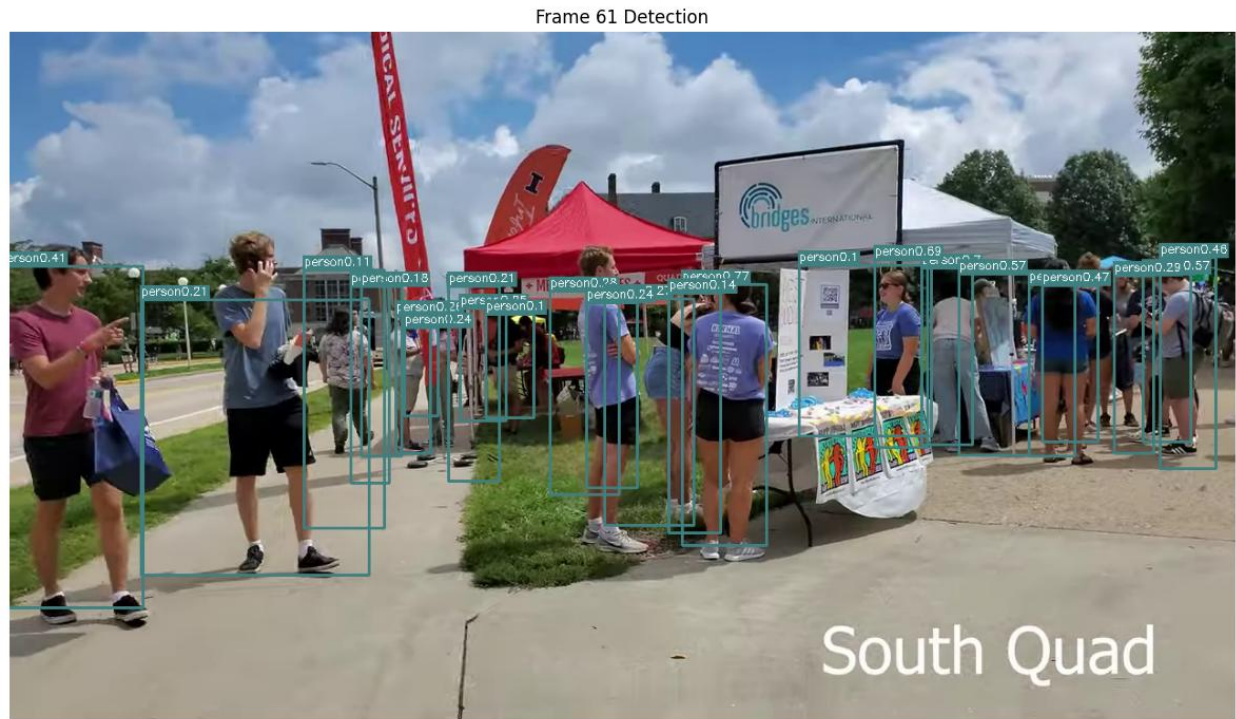
1. **My best mAP value on Kaggle:** 0.52211
2. **Did you upload the final CSV file on Kaggle:** Yes, the team name is “Libin Wang&Licheng Xu”.
3. **My final loss value:** 2.49974
4. **What did not work in my code (if anything):** N/A
5. **Sample Images from my detector from PASCAL VOC:**



Extra Credit for YOLO:

Part(1): Video detection

We downloaded a video in YouTube and tried to do object detection using our Resnet50-based YOLO model. We used opencv library to convert the video in mp4 format to 2721 frames and did object detection for each frame to generate recognition frames. Below is the result of object detection for frame 61 of the video:



We finally converted all 2721 frames after recognition back to mp4 video format with the name "UIUC_Quad_day_yolo".

Part(2): Better pretrained network

Our original YOLO model used Resnet50 as a pre-trained model. In order to get better test performance, we chose to use the Resnet101 model instead of the original Resnet50 and train the new YOLO model.

We first defined the new function "resnet101(pretrained=False, **kwargs)" in the resnet_yolo.py file. Compared to the function "resnet50(pretrained=False, **kwargs)", Resnet101 increases the number of blocks in layer3 from 6 to 23; everything else remains the same.

Considering the rise in the number of trainable parameters of the model, we increase the learning rate to 5e-3 and increase the batch size to 24. At the same time, due to arithmetic and time constraints, we reduce the number of training rounds to 35 and decrease the learning rate to 1/10 in rounds 15 and 25.

The final test mAP of the model in round 35 is 0.51252... , and the test loss was 2.49339. It is worth noting that the model showed a potential tendency to overfitting after epoch 25: the test mAP decreased a bit, but so did the test loss. We believe that by decreasing the initial learning rate, increasing the L2 regularization of the optimizer appropriately, and refining the learning rate decay strategy, the new model's test mAP at epoch 50 can be improved by 0.05 compared to the original model.

Part(3): Alternative detection loss function

We changed all four loss functions to alternative implementations (see `extra_torch.py`). For the classification loss, we used label-smoothed cross-entropy. For both no-object and object confidence, we applied binary cross-entropy with logits and binary cross-entropy, respectively. For box regression, we employed Smooth L1. These modifications were intended to make the model focus more on hard and false-positive examples, improving robustness to outliers and label noise. However, possibly due to the architecture or other factors, this implementation did not yield the expected results: after several epochs, it still produced a mAP of 0. We found that the revised loss functions caused the no-object loss weight to become so dominant that the model's performance suffered. Therefore, we temporarily disabled the no-object loss and trained for several epochs, which yielded performance similar to that of the original model in first several epochs, then after 15 epochs, the performance decrease due to one loss function has been turned off. Then we test with the appropriate new `lambda_noobj` and find out that it should be smaller than the previous one, and we find out that 0.005 works fine.